确定性多模态时间对齐与低延迟推理平台

提交版设计报告 • 2026-10-29

目录

阅读说明 2

1 概述与应用背景 2

2 系统总体架构 3

3 核心设计实现 5

4 硬件实现与资源/时序 7

5 验证与测试 9

6 大模型协作与技能包 13

7 失败分析、局限与复现说明 14

附：一句话总结 17

基于 PYNQ-Z2（XC7Z020）的定点注意力引擎与 10 ns 时间戳前端

第九届（2026）全国大学生嵌入式芯片与系统设计竞赛 • FPGA 创新设计赛道 • AMD 自主选题

阅读说明

本文所有数字都指向仓库内的一个文件。凡不能从本仓库复现的，都在正文里显式标注为「不可复现」或「未测量」，并在第 7 节汇总。

文档	内容
`metrics/metrics.csv`	全部结论性数字的总索引，一行一条，附证据路径与限制
`metrics/{latency,resource,timing,accuracy,cpu_baseline}.csv`	分项明细
`build/reports/*.rpt`	Vivado 原始报告，`metrics` 中的资源与时序数字由它转录
`sim/run/run_attn_m{2,3,4}.sh`	一条命令复现逐位验证
`BITSTREAM_MANIFEST.md`	哪份位流产生了哪组数字，md5 与参数陷阱
`report/ai_collab.md`	大模型协作的完整记录与纠错轨迹
`skill/`	可复用技能包（`pynq_probe` + `golden_compare.py`），50 项离线测试，`sh skill/tests/run_tests.sh`

本报告不包含学校、指导教师信息（按赛制要求）。

---

1 概述与应用背景

1.1 问题

多模态生理信号（脑电 EEG、眼动、面部动作单元 AU、皮肤电 GSR）的融合模型有一个工程上很少被认真对待的前提：**四个模态的样本必须在时间上对齐**。实际采集中，四台设备的采样率、上电时刻、传输延迟、缓冲区抖动互不相同；一个 4 s 的分析窗口在四路数据上的真实起点可以差到几十毫秒甚至更多。现有做法通常是在 PC 上事后插值对齐——离线可用，在线不可用，而且对齐误差从未被量化过。

本项目要回答的问题不是「能不能识别情绪」，而是一个更窄、更确定的工程问题：

> 在一个嵌入式平台上，能否把「多模态数据的时间戳、对齐、融合推理」做成一条**确定性、可逐位验证、延迟可算**的通路？

1.2 为什么用 FPGA

三个理由，按重要性排序：

1. **逐位可验证**。浮点模型换一个库、换一个 CPU、换一个编译选项就变一个结果。定点 + 定拍数的硬件实现，在同一份输入上给出**同一串比特**。本项目的验收标准就是逐位相等（28 个数组 / 5,504 个字），而不是「误差小于多少」。

2. **延迟可算**。引擎耗时是 2,079,001 个时钟周期，不是「大约 20 毫秒」。这个数在仿真里数得出来，在板上也能被计数器复现。

3. **时间戳精度**。硬件计数器可以做到 10 ns 量化，且不随 CPU 调度抖动。

1.3 本作品的定位（必须明确）

本作品是「**确定性多模态时间对齐 + 低延迟推理平台**」，不是「情绪识别产品」。这个界定不是谦虚措辞，而是由数据决定的：

项	事实	出处
三分类机会水平	33.3 %	—
本设计的实际 macro-F1	0.34–0.40	`metrics/accuracy.csv`
四路真实传感器	**未接入**。板上时间戳来自内部节拍`SRC=0xF`	第 7 节
功耗	**未测量**（无功率计、无 PMBus）	第 7 节

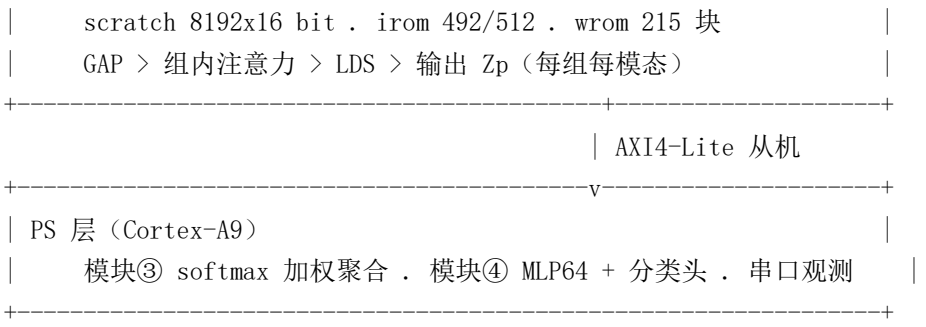
因此本报告**不主张**「实时识别情绪」「四路已同步」「在线校准有效」。它的价值在平台：一个把「时间戳→对齐→定点融合→逐位验收」闭环做通的、可复用的硬件底座。第 7 节把这一点写成了失败分析。

---

2 系统总体架构

2.1 分层

```
+-------------------------------------------------------------+
| 采集层（本作品未实现，接口已就位）                          |
|     EEG 62ch@256Hz . 眼动 . AU . GSR . 外部同步沿（DRDY）   |
+-----------------------------------------+-------------------+
                                          |
+-----------------------------------------v-------------------+
| PL 时间戳层  ts_top / ts_core                               |
|     48 bit 计数器 @10 ns 计数 = 32.58 天量程                |
|     每模态首次沿 > 打戳 t_m；t_m 补偿；e_m 存而不扣         |
+-----------------------------------------+-------------------+
                                          | ts_m[47:0]
+-----------------------------------------v-------------------+
| PL 特征层  eeg_fe_top（EEG 频带能量）                       |
|     62 通道 x 5 频带，逐样本 87 拍，256 Hz 下占 1.4 %@100 MHz |
+-----------------------------------------+-------------------+
                                          | int16 特征
+-----------------------------------------v-------------------+
| PL 推理层  attn_axi --> attn_core（M=2/3/4）                |
```



2.2 模块清单

文件	行数	职责
src/rtl/attn_core.v	1,083	定点注意力引擎，M=2/3/4 共用一份 RTL
src/rtl/attn_axi.v	320	AXI4-Lite 从机、scratch 窗口、START/DONE 协议
src/rtl/eeg_fe.v	672	EEG 频带能量前端（128 深 FIFO + 累加器）
src/rtl/eeg_fe_top.v	332	前端 AXI 包装与寄存器
src/rtl/ts_top.v	422	时间戳子系统顶层
src/rtl/ts_core.v	209	打戳、τ 补偿、ε 存储
src/rtl/tau_comp.v	85	τ 补偿
src/rtl/log2_cordic.v	280	对数（CORDIC），EEG 对数方差
src/rtl/align_buf.v	149	对齐缓冲
src/rtl/win_stats.v	148	窗统计
src/rtl/sdpram.v / frac_div.v / strobe_gen.v	82 / 66 / 73	存储、定点除法、节拍分频
src/rtl/audio_top.v 等 4 个	532	I2S 音频支路（本项目未纳入验收口径）
合计	**4,453**	17 个 Verilog 文件
src/model/fixpt.py	439	定点参考模型（逐位标准答案）
src/model/export.py	426	权重与测试向量导出

2.3 PS/PL 划分的依据

不是凭直觉分的，是量出来的。分类头（模块③④，MLP64 + softmax 聚合）留在 PS：

实现	耗时
PS 纯 C（Cortex-A9）	0.242 ms
PL 纯计算	0.167 ms
PL 计算 + CPU↔PL 搬运	**1.004 ms（5.01×）**

PL 算得快 1.45 倍，但把 1,024 字的往返搬运算进去就慢 5 倍。所以只有计算量大、数据宽度低的注意力引擎

进 PL，小层留在 PS。（数据来源为设计笔记，属**二次引用**，见`metrics/cpu_baseline.csv` 的`status` 列。）

2.4 时钟域与复位

单一设计时钟`clk_fpga_0`，来自 PS7 的`FCLK_CLK0`。**这一点很关键：**

- 块设计把该时钟约束在 **200 MHz**（`PCW_FPGA0_PERIPHERAL_FREQMHZ {200}`），所以 Vivado 的默认时序报告是按 200 MHz 出的，而且**不收敛**（见 4.2）。
- 板上应用在启动后把`DIVISOR0` 改成 10，实际运行在 **100 MHz**。
- 板上实测时钟读数 100.00 MHz，引擎 2,079,001 拍对应 20.790 ms——三者自洽。
- AXI 总线、三从机互联、EEG 前端、时间戳核全部在同一时钟域内，无跨时钟路径。

---

3 核心设计实现

3.1 定点算术契约

逐位验证的前提是**算术契约写清楚**，而不只是「用定点」。

对象	格式	位宽
输入特征	int8（缩放 2^{-10}，四个模态统一）	8
权重	int8	8
累加器	有符号 Q 定点，乘积 42 bit	42
激活	int16，右移还原	16
日志/输出	int16	16

EEG 前端另有独立契约：输入 int16 `<<6` → Q3.21；系数 Q3.15（18 bit）；状态/输出 Q3.21（24 bit）；乘积 42 bit。三个在 Verilog 里必须显式处理的坑：

1. **有符号数混进无符号表达式会先零扩展**——负数变成大正数。
2. **乘法结果宽度取表达式宽度，不是操作数之和**——静默截断。
3. **72×72 大位宽乘法必须拆拍**——200 MHz 下不拆实测 WNS −10.0 ns。

这三条都写在对应模块的头注释里，因为它们的失败方式是「算错了但不报错」。

3.2 注意力引擎（M=2/3/4）

一份 RTL 支持三种模态规模，通过**参数与权重 ROM 内容**切换，而不是三份代码：

规模	模态	权重字节	权重块	指令数	scratch
M=2	e+o	29,984	115	182	5,376
M=4	e+o+a+s	55,824	215	492	8,192（满）

设计要点：

- **指令驱动**：运算序列存在`irom`（492/512 项），引擎按指令流执行 GAP、组内注意力、LDS、输出，而不是硬连线。换 M 只换`irom` 与`wrom` 内容。
- **wrom 统一 44 块**：地址空间固定`0:131071`，不随 M 变化——这让三份位流的资源报告可比，也避免了「换 M 忘改深度」这类错误。
- **因果掩码**：注意力矩阵的上三角被屏蔽，保证第 i 个位置只能看到 j ⩽ i。
- **scratch 是硬约束**：M=4 时 8,192 / 8,192 已满。这不是性能问题，是**能力边界**：在加深`mem` 与`irom` 之前，M=5 不可能实现。第 7 节把它列为已知限制。

一个真实的陷阱（值得单独写）：`attn_axi.v` 的 scratch 深度是 Verilog 参数`UM`。**Vivado 不会用块设计去覆盖 Verilog 参数的缺省值**——缺省值就是进位流的值。M=4 的位流必须用`UM=12`，但树里的缺省是 8。用 8 重建 M=4，会综合通过、时序收敛、正常启动，然后**静默地算错**。这个坑是 2026-10-27 实际踩到的，处理方式：

- 把`UM` 改成宏`` `ATTNAXI_UM``（缺省 8），M=4 构建必须定义 12；
- 写`BITSTREAM_MANIFEST.md` 记录每份位流的`UM`；
- 用逐位测试台作为最终防线——它不关心你是怎么配错的，只报`MISMATCH`。

3.3 时间戳前端

- 48 bit 计数器，10 ns 计数，量程 32.58 天（不溢出）。
- 每模态数据到沿打一次戳 t_m；τ_m 是固定补偿偏置；ε_m 是实测抖动，**只存不扣**。「只存不扣」是刻意的：扣掉会掩盖误差，存下来才能在第 7 节的失败分析里被引用。
- 寄存器`SRC` 标识时间戳来源：`0xF` = 内部节拍。**这正是「四路未真正同步」的自证**：硬件自己说了它用的是内部节拍，报告里就不能写成「已同步」。

3.4 EEG 前端

62 通道 × 5 频带（delta/theta/alpha/beta/gamma），逐样本 87 拍。256 Hz 采样率下占用率 62×256×87 / 100 M = **1.4 %**（100 MHz）。一个 128 深 FIFO 吸收 PS 写入抖动；累加器按 (通道, 频带) 索引，**采样顺序无关**——这让它对接真实 DRDY 时不需要按顺序写入。

寄存器：`CTRL`、`STATUS`、`WIN_CNT`、`BEAT_CNT`、`SAMPLE`、`RES_INDEX`、`RES_DATA`、`OVF`，外加两个身份字`BUILD=0x5EED0001`、`GEOM=0x003E0500`（=NCH 62、NB 5）。身份字的作用是让驱动能在跑之前确认「板上是不是我以为的那份前端」。

3.5 AXI 从机与装窗协议

偏移	名	说明
0x0000	CTRL	W，bit0 = START（单次；busy 时忽略）
0x0004	STATUS	R，bit0 = busy，bit1 = done（粘滞）
0x0008	ID	R，0x41540001（'AT'）
0x000C	DONE_CLR	W，清粘滞 done
0x0100–0x40FF	scratch	RW，8192 × 16 bit

按 32 bit 拍写：第 W 个拍对应字节地址0x0100 + 4W，一拍拆成两个 16 bit 字（2W 与2W+1）写入 scratch。窗口 0x0100–0x40FF 共 16 KB = 4,096 拍 = 8,192 个 16 bit 字，与UM 无关——**窗口大小固定，深度由UM 决定**，这正是 M=4 用错UM 时不会越界、只会算错的原因。参数：T=4、D=16、UM=8（M=2/3）/ 12（M=4）、AW=16。

done 粘滞而不自动清零，是为了让 PS 的轮询不会漏掉一个很短的完成脉冲——代价是 PS 必须显式写 DONE_CLR。裸机固件里这一条写错会导致第二次 START 立刻返回。

3.6 关键路径：为什么 200 MHz 不可达

M=4 的最差路径（build/reports/worst_m4.rpt）：

```
acc0 的 FDRE.Q  -->  lds_p0r 的 DSP48E1.B[5]
数据路径 1.917 ns（其中布线 1.439 ns），逻辑级数 0
DSP48E1 setup 要求 3.707 ns  < 吃掉了 5 ns 预算的 74 %
```

问题不在组合逻辑，而在于**累加器寄存器到 DSP 输入之间没有流水级**，而 DSP48E1 的输入端建立时间本身就是固定的 3.7 ns。要在 200 MHz 下收敛，必须把累加器拆成两级（多一拍延迟，拍数会变，逐位标准答案也要跟着重新导出）。

没有做这个改动的理由：改流水级会改变拍数，从而让已发布的 2,079,001 拍、板上实测、三份位流的 md5 全部失效。在距提交 6 天的时点上，把可用设计换成「理论更快的不可用设计」是错误的取舍。**运行在 100 MHz，+4.264 ns 余量，留有记录。**

---

4 硬件实现与资源/时序

4.1 器件与工具

项	值
器件	XC7Z020-1CLG400C（PYNQ-Z2）
工具	Vivado / Vitis 2024.1
赛制推荐	2026.1 或 2025.2
为什么用 2024.1	全部历史证据（位流、板上实测、逐位日志）都由 2024.1 产生。换工具链会让「位流 md5 与实测对应」这条证据链断开

所有构建脚本通过XILINX_VIVADO / XILINX_VITIS 环境变量解析工具路径，不写死盘符；在 2025.2/2026.1 上运行需要重新验证（**未测试**，见 7.1）。

4.2 资源（整机块设计，routed）

规模	Slice LUT	Slice Reg	BRAM 块	DSP48E1
M=2	6,649 (12.50 %)	9,537 (8.96 %)	44 (31.43 %)	20 (9.09 %)
M=3	7,135 (13.41 %)	9,559 (8.98 %)	44 (31.43 %)	20 (9.09 %)
M=4	6,995 (13.15 %)	9,566 (8.99 %)	44 (31.43 %)	20 (9.09 %)

来源：`build/reports/util_m{2_e4,3,4}.rpt`，`metrics/resource.csv`。

两点必须说明，否则容易被误读：

1. **M=4 的 LUT 比 M=3 少**（6,995 < 7,135）。不是笔误：M=4 的引擎把四路模态压缩进同一组共享的定点通路，而 M=3 的指令流里有更多互不相交的中间活寄存器被 phys_opt 复制。2026-10-29 的干净重建把 M=3 从「暂定」变成了确定值：重建给出 7,135 LUT / 9,559 FF / WNS −0.656 / 197 失败端点，与主表记录的交付`attn3_e4.bit` 完全一致。此前仓库里的`util_m3.rpt` 是 2026-10-24 的检查点（7,071 / 9,485 / −0.639 / 239），描述的不是交付位流，已替换。
2. **54.5 KB 是权重 ROM，不是 BRAM 总量。** 44 个 BRAM 块≈ 198 KB，其中权重占 54.5 KB（8.65 %，另一套分母）。这两个数在不同口径下出现，混用会得到「BRAM 只用了 8.65 %」这种错得离谱的结论。

OOC 口径下`attn_axi` 单独综合：2,708 LUT / 1,839 FF / 38.5 BRAM / 8 DSP。**OOC IP 与整机 BD 是两个口径**，端点数和 LUT 数都不能直接比。

4.3 时序

位流	WNS @200 MHz	失败端点	WNS @100 MHz（签核点）
`attn2_e4.bit`	−0.919 ns	843 / 25,249	**+4.081 ns**
`attn3_e4.bit`	−0.656 ns	197 / 25,273	**+4.344 ns**
`attn4_e4.bit`	−0.736 ns	274 / 25,366	**+4.264 ns**
`ts_e4.bit`	+0.083 ns	0	（100 MHz 余量更大）
A1 三从机基线（无 ε）	+0.002 ns	0 / 23,719	—

保持时间：M=4 `WHS = +0.037 ns`，0 个失败端点。M=2 `+0.040`，M=3 `+0.016`。

口径说明（必须一起引用）：块设计按 200 MHz 约束，所以 Vivado 默认报告是 200 MHz 的，且**全部为负**。交付位流实际运行在 100 MHz，在该点有 4.1–4.3 ns 余量、零失败端点。100 MHz 的数字是在 routed 检查点上按 10 ns 重新报告得到的，**不是「200 MHz 的值加 5」**——那只有在单时钟域且路径完全相同时才成立，本项目选择重新报告失败路径。复现命令见`build/reports/README.md`。

反向证据也保留着：`ts_e4` 的同设计默认`impl_1` 结果是−0.012 ns、有失败端点；交付位流来自调优后的实现。保留这一行是为了说明报告没有只挑好数据。

4.4 位流清单

位流	模式	大小 (B)	md5
attn4_e4.bit	M=4（生产）	4,045,674	1e2555571c380d8fbaa2324141023e6f
attn3_e4.bit	M=3	4,045,674	a607316a7041fdb9798c139bff430a6c
attn2_e4.bit	M=2	4,045,674	d9fd5e4565f20f931e51df18aa89da48
ts_e4.bit	时间戳核	4,045,673	4fa3518fcb61ad5d91d8379001909806
BOOT.BIN	M=4 裸机启动	4,231,752	93f8caee4a2e4b365622cfa0dee230d2

位流不提交到仓库（见.gitignore），随提交包分发。**md5 标识的是交付的那串字节本身**：表里的每个数字都绑定到这一份。但 md5 不相等**不能直接推断成「不是同一份设计」**——2026-10-29 的重建就给出了反例：M=2 与 M=4 抹掉头部日期域后 md5 相等；M=3 有 57 字节配置数据不同，而 LUT、FF、BRAM、DSP、WNS、WHS、失败端点数全部一致，交接文件的关键部分逐字节相同（§5.8）。所以规则是：**md5 对不上，就必须把它解释掉**——报告一致、交接一致、结构 gate 通过——而不是拿 md5 当判决书。

4.5 构建流程

```
build/build_bd_all.tcl        > 块设计 bd_all
build/build_bd_all_m*_impl.tcl > 综合/实现/写位流
build/report_a1_util.tcl      > 从 routed .dcp 抽资源与时序
board/baremetal/build_elf.ps1     > 裸机 ELF
board/baremetal/build_bootbin.ps1 > FSBL + bit + ELF > BOOT.BIN
```

关于脚本布局：这些 Tcl 原本按开发者树布局编写（需要ip_repo_all/、work2/out/ 作为兄弟目录），而提交仓库是扁平英文树。两条对策都已落地：

- build/make_tree.ps1 只用仓库里的文件把工作树重新铺出来，并改写两行（MEM_DIR 的绝对路径、attn_axi.v 的UM 缺省值），仓库本身不动；
- 每个build_*.tcl 开头有显式 guard，布局不符时**立即报错退出**，而不是四十行之后在 Vivado 内部失败（静默回退到过期 IP 是这个项目花了一周追的故障）。

这套流程 2026-10-29 在一台没有开发者工作树的机器上跑通了，结果见 §5.8 与BITSTREAM_MANIFEST.md。

---

5 验证与测试

> 本文全部数字对应提交截止（**2026-11-04 18:00**）前的最终版本；标为「未测量/ 未验证」的项是**真的没做**，没有用资源或理论值反推的代填数字。

5.1 逐位验证（仿真，可完全复现）

```
cd sim/run && ./run_attn_m4.sh
```

规模	数组	字	结果
M=2	14	3,200	ALL PASS
M=3	21	4,224	ALL PASS
M=4	28	5,504	ALL PASS

比较对象是`src/model/fixpt.py` 导出的金标准向量（提交在`sim/vectors/<mode>/`）。判定是**逐字相等**，没有任何容差。测试台同时报出引擎拍数（466,157 / 1,124,691 / 2,079,001），使「延迟」与「正确性」在同一份日志里。

这条链路是自包含的：不需要板子、不需要模型权重、不需要 AFFEC 原始数据、不需要联网。只需要 Vivado。这是「可复现性」最直接的证据。

向量重生成（可选）：`AFFEC_CACHE=... ./regen_vectors.sh m4`。重生成是**确定性**的，产出的文件与提交的向量逐字节相同。

5.2 板上逐位验证

- 三次重复 × 28 数组 × 5,504 字，全部一致（`board_repeat_results.json`）。
- 输入是**同一份固定输入重复 3 次**，所以它证明的是「可重复」，不是「覆盖充分」。
- 两条独立启动路径互证：JTAG 重灌 ELF 与 SD 卡冷启动，同一份`attn4_e4.bit`，都是 28 / 5,504 逐位一致⇒「换启动方式没有改变功能」有证据，不是推断。
- 单次计时 50 次：中位 21.644 ms，p99 21.646 ms，极差 2 µs。

5.3 延迟

规模	拍数	理论 @100 MHz	板上实测	差异来源
M=2	466,157	4.662 ms	4.686 / 4.717 / 4.742 ms	三次独立运行
M=3	1,124,691	11.247 ms	11.33 ms	宿主轮询粒度
M=4	2,079,001	20.790 ms	20.89 ms（10-28 复现 20.88）	宿主轮询粒度
M=4 裸机	2,079,001	20.790 ms	**20.790 ms**	寄存器轮询，粒度更小

端到端（一个 4 s 窗口）：

路径	合计	拆分
Python 宿主	36.48 ms	装载 8.41 + 引擎 20.88 + 头部读回 6.72（合计 36.01，剩余 0.47 ms 未细分）
裸机，无宿主	**21.650 ms**	装载 1,636 字 0.189 + 引擎 20.790 + 读回 1,024 字 0.113 + PS 模块③④ 0.555

裸机路径占 4 s 窗口的 **0.54 %**。这个数不含验收用的 5,504 字回读比对——那是验货动作，不是生产动作。

口径警告：M=3 → M=4 引擎是 1.86×，但端到端只有 0.86×（反而快 14%，因为宿主侧的固定开销被摊薄）。引用「加速比」必须同时给这两个数，否则会得出自相矛盾的结论。

5.4 同板 CPU 基线

「PL 快多少」如果没有同板分母，就只是自说自话。这一节补上分母——而且它给出的**不是**一个漂亮的加速比，是一条对这个设计的否证（2026-10-29 实测，见`board/baseline/`，运行记录`cpu_ref_20261029_152542.txt`）。

做法不是另写一份注意力实现，而是让 **Cortex-A9 执行同一条指令流**：492 条指令、同一份 INT8 权重、同一组 requant 参数、同一张 exp 表、同一份测试向量，同一个 8,192 字 scratch 布局。`gen_cpu_ref.py` 直接把`sim/vectors/m4/*.mem` 转成 C 表（不重新量化任何东西），`cpu_ref.c` 按`attn_core.v` 的状态机逐操作码复刻。

	PL @100 MHz	**Cortex-A9 @650 MHz**	**比值**
模块①②引擎	**20.790 ms**	**7.470 ms**（5 次中位；min 7.452 / max 7.473）	**A9 快 2.78×**
同口径拍数	2,079,001	≈ 4.85 M	PL 少 2.33×
对`exp_*.mem` 逐位一致	是	是，22/22 块	—

结论和预期相反：在这条负载上，PL 比同板的 A9 慢 2.78×（20.790 / 7.470）。这个数站得住的前提是两边跑的是**同一个工作量**：指令流相同⇒ 操作数顺序、舍入位置、什么算一个窗口全都相同，实现差异被排除，剩下的只有执行速度。这也是为什么用「解释器」而不是「另写一版 C 注意力」——后者任何一处舍入不同，都会变成一个假的分母。

即：**PL 每拍干的活是 A9 的 2.33 倍，但时钟低 6.5 倍（100 vs 650 MHz）；2.33 < 6.5，净输**。引擎用 20 个 DSP48E1（器件的 9 %）跑约 1.5 M 次 MAC，合每周期约 0.7 次 MAC——并行度远没用满，瓶颈是累加器链而不是 DSP 数量（与 §5.6 的 200 MHz 缺口同源）。按这个算式，把累加器加一级流水、收住 200 MHz，得到 10.40 ms，**仍然落后 1.39×**。所以这条基线的意义不是「加速比的分母」，而是一条**否证**：在当前实现下，「把大层搬进 PL 是为了更快」这个动机不成立。要它成立，得先提高每拍 MAC 数（多路 DSP 并行 + 压低 requant 开销），而那会改掉全部已验证的拍数与向量。

与 §2.3 的关系：2.3 节的 1.45× 是分类头**纯算术**的 PL 优势（0.167 vs 0.242 ms，标注`secondary_citation`），那是一个小而密集的矩阵乘；这里的 2.78× 反超发生在整条指令流上。两者不矛盾：小层 PL 快、大层 PL 慢，说明瓶颈不在「PL 还是 CPU」，而在**这份 RTL 的每拍吞吐**。

公平性前提（与数字同页，不放进脚注）：

- CPU 侧是**标量解释器**，`-O2`，单线程，无 SIMD。本器件的 A9 只有 VFPv3-D16、无 NEON，但手写展开的 int8 点积仍会快于这个解释器⇒ **2.78× 是 A9 优势的下限**，换更强的 CPU 实现只会把这个差距拉大，不会缩小。
- 两边都只含模块①②，都不含 AXI 装载 0.190 ms 与回读 0.113 ms——这两步在 CPU 路径上形态不同，拉进来比较会把口径搅乱。
- D-cache 开、权重在 DDR，与生产应用同姿态；CPU 频率取自 BSP 的`COUNTS_PER_SECOND`（325 MHz = 核频的一半），不是假设值。

复刻过程中有三处「照字段字面执行就会错」的地方，记在这里供复核：`N` 字段只有 6 位、64 行编码为 0，RTL 靠`o_N − 1` 回绕跑满；`PE` 是原地操作（`S_EXEC` 只取`o_B`，`A` 不是操作数）；`RESGAP` 的移位量取`LG2T6` 局部参

数，而非指令里的SH 字段。三处若按字面执行，分别表现为整段模态丢失、PE 加到错误的块、时间均值变成时间求和（整 4 倍）。

5.5 精度

协议	结论	95% CI
80 折 pooled，e+o+a − e+o	**+2.28 pt**	[+0.55, +3.74]（折簇 bootstrap）
同上，另一套 CI	+2.28 pt	[+1.19, +3.37]（20 折配对 t）
64 被试 bootstrap，AU 主效应	+2.287 pt	[+1.466, +3.122]
64 被试 bootstrap，C（加 AU）	+2.451 pt	[+1.506, +3.405]
三种子复现 C	+2.28 / +2.30 / +2.10	3/3 排除 0，极差 0.19 pt
再加 GSR（S）	+0.01 / −0.44 / +1.24	**变号⇒ 种子依赖**

两个估计量不能混用：+2.28 是**折均值**，+2.451 是 **pooled OOF**。两者都叫「AU 的增益」，但不是同一个统计量，并排出现时必须加脚注。

四靶之间有真实的异质性：交互项在四靶合计上含 0，但f_emotion_v +1.95 与f_emotion_a −2.72 符号相反且各自排除 0。写「无交互」是错的。

5.6 量化误差

量	值
float 模型 macro-F1	0.36811
权重 INT8（软件算术）	0.37003
硬件算术	**0.36695**
float vs 硬件一致率	**99.61 %**
四模态输入饱和计数	**0 / 0 / 0 / 0**（1.26 M / 98 K / 207 K / 98 K 元素）

结论：在这一个折、这一个靶上，量化**没有**造成精度损失（硬件算术甚至略低于权重 INT8，但在噪声范围内）。输入侧零饱和说明 INT8 量化没有削顶——这是量化风险里唯一被检查过的一项。

边界：这是单折、单靶、1,016 试次。它证明「这条硬件通路端到端能给出与浮点一致的预测」，不证明「在所有折上都一致」。

5.7 回归一致性

- 旧的 80 个模型，每折 F1 重现到1e-6（original_fold_manifest.json）。
- 稳定折清单匹配、旧 80 折分数复现、τ 元数据不匹配被拒绝——全部为真（verification.json）。
- 这一条的意义：**改了代码之后，旧的结论还成立**。没有它，「修 bug」和「改结果」就分不开。

5.8 复现方法

见`README.md` 的 Quick start。非板端步骤在干净副本上可跑通（Day 5 演练）；板端需要 PYNQ-Z2、SD 卡、USB 串口。

位流重建（2026-10-29 实测）：在无开发者工作树的机器上，只用仓库文件跑`build/make_tree.ps1 -Mode <m>` →`build_bd_all.tcl` →`build_bd_all_m*_impl.tcl` →`write_bit_attn*.tcl`（Vivado 2024.1）。三档都跑通：M=4 与 M=2 产出的位流与已发货那份**只差构建日期那几个字节**（M=4 五个、M=2 七个，全在`.bit` 头部 82–106 字节内）；抹掉该域后两边 md5 分别同为`7f2a3fea…`与`4cb32e0c…`，即配置数据逐位相同，`.rpt` 与`hwdef.xml` 除日期和路径外完全一致。M=3 重建给出 7,135 LUT / −0.656 ns / 197 失败端点，与主表记录的交付`attn3_e4.bit` 完全吻合（此前仓库里的 M=3 报告是更早的检查点，已换掉）；但它的位流**不是**逐字节相同的——63 字节不同，其中 57 字节在配置数据区。判定为同一设计的不同布线解：凡逻辑变化必然牵动的量（LUT、FF、BRAM、DSP、WNS、WHS、失败端点数）全部一致，`hwdef.xml`、`all_bd.bda`、`ps7_init.*` 逐字节相同。交付的 M=3 位流出自一份不在仓库里的实现脚本（`build_bd_all_e4_impl.tcl`），出处缺口是这条差异最经济的解释，但这只是解释而非证明。注意`.bit` 的 md5 **不可能**直接相等，比对须先抹掉头部日期域，见`BITSTREAM_MANIFEST.md`。

---

6 大模型协作与技能包

完整记录见`report/ai_collab.md`，此处给结论。

6.1 协作方式

本项目的硬件部分由人机协作完成：人负责定义契约（算术格式、寄存器映射、验收判据），模型负责实现、审查、以及**在被数字反驳时改口**。协作的价值不在「生成代码快」，而在**把「我以为是」变成「日志里写着」**。

6.2 三个真实的纠错案例

案例一：`UM` 参数漂移模型的建议是「用块设计覆盖参数」，这个建议是**错的**——Vivado 不从 BD 覆盖 Verilog 参数缺省值。被位流 md5 与逐位测试台否定后，改为宏 + 清单 + 测试台三道防线。

案例二：盘符硬编码初版审计口径把注释、文档、生成的 Vivado 树都算进去，得出 4,552 处「硬编码」，这个数字没有可操作性。后来把口径收窄到「可执行 + 工程路径 + 非注释」，真正的必修面从上千处降到 16 个必需文件的 126 处，修完为 0。**教训：审计的第一件事是定义什么不算。**

案例三：`-d` 传字符串在 Windows 上会崩 `xvlog.bat -d MEM_DIR="\"../vectors/m4\""` 报`Can not find file: "\"../vectors/m4\""`。模型一开始坚持这是转义问题，连续三次改写引号都失败；改为「生成一行`memdir.vh` 编译在最前面，用`` `ifndef `` 让 RTL 缺省失效」后一次通过。**教训：工具的失败模式要用最小复现去否证，不要靠推理修引号。**

6.3 `skill/` 技能包

技能包做两件事。**第一件是把软件和位流对齐**，第二件是把数值对齐。

一、`pynq_probe`：先确认 FPGA 里装的是什么，再跟它说话。

所有环节中反复出现的失败不是逻辑写错，而是**软件在跟一个不是它写的设计通信**。症状都长得像逻辑 bug：写落到别的块里、状态位读回成数据位、结果块固定偏移、窗口悄悄越过 AXI 孔径（AXI 从机在真实范围之外照样回 OKAY）。三个离线步骤在第一次 MMIO 访问之前就把这类错挡住：

- `hwh.py` 读`.hwh`/`.xsa`，给出 IP 清单、孔径、地址顺序、时钟频率。它同时说明**这个文件查不到什么**：经 `` `define `` 定的 Verilog 参数不会写进交接文件，所以两份`.hwh` 可以逐字节相同而算出的东西不同——本项目的`UM` 正是这样。测试里断言了`"UM" not in attn.params`，让这个结论不会从文件里悄悄消失。
- `regmap.py` 用一份 JSON 描述寄存器映射，再对着交接文件校验：寄存器或窗口越出孔径、区域重叠、字段重叠、字段宽于寄存器、`modtype` 不符、参数不符、实例根本不在 FPGA 里，逐条拒绝。`resolve()` 对没通过校验的描述**拒绝**给出地址——不可信的地址比没有地址更糟。
- `capture.py` 逐位比较结果块：长度必须相等，值必须相等，不给容差，报第一个不同的字的下标。**短采集即使每个字都对也判失败**，这一点多数脚本会写错。
- `overlay.py` 是板上模块：先比对身份寄存器再放行，提供装窗/读窗/启动等待，处理 DONE 清零与 BUSY 假掉。

边界（自己写的，不是别人挑的）：前三个模块有 50 项离线测试，**`overlay.py` 没有在硬件上跑过**，是按 PYNQ 3.x API 写的，用它得出的数在板子上验证之前不能采信。这句话写在`skill/README.md`、`skill/validation.json` 和模块 docstring 三处。

二、`golden_compare.py`：NPZ 数值工件的逐位比对。

默认要求值、形状、dtype、键集全部相等；缺数组、多的数组、形状变化、dtype 变化都判失败；object 数组拒绝反序列化；记录哈希把结果绑定到工件。浮点比较必须显式给 `--atol`，相对容差恒为 0（相对容差会让大数的错被小数的错掩过去）；声称整数逐位相等时不允许任何容差。

两者的来源都是这个项目真实踩过的坑：有符号/无符号十六进制不匹配、过期的生成 ROM、缺失的输出张量、过期的综合 IP 检查点、以及最贵的那一次——软件按默认`UM` 写窗口，位流里却是另一个深度，逐位比对报`MISMATCH` 而不是崩溃。

它不能证明什么：通过任何一项**都不**证明时序收敛、物理同步、模型精度、电气安全，也不证明加载的是预期位流。寄存器映射校验只证明**描述**与交接文件一致；真正把数字绑定到设计的，只有运行时的身份寄存器比对和位流 md5。

复现：`sh skill/tests/run_tests.sh`（离线，不需要板子、不需要 pynq、不需要网络）。

6.4 可复用经验

1. **让「错」有一个会响的测试台。**逐位比对的成本很低，但它把「静默算错」这个最难发现的失败模式变成了一行`MISMATCH`。
2. **区分「量出来的」和「推出来的」。**本报告里 200 MHz 与 100 MHz 的时序、20.790 与 20.89 与 21.650 三组延迟，都标了口径；不标就会被当成矛盾。
3. **保留反例。** `ts_e4` 的默认实现失败、M=2 是 200 MHz 下最差的一档、S 臂三种子变号——这些都留在表里。只放好数据的报告，评委会自己去找坏数据。

---

7 失败分析、局限与复现说明

这一节是报告里最重要的部分。本作品的**工程证据强、应用闭环弱**，下面逐条写清楚弱在哪里、为什么、以及如果继续做应该怎么改。

7.1 未达成的目标

#	目标	状态	原因
1	四路真实传感器接入	**未实现**	无 EEG/GSR/眼动/相机前端硬件。板上时间戳来自内部节拍`SRC=0xF`
2	外部同步误差测量	**未实现**	需要同步台架 + 示波器/逻辑分析仪，本项目无此条件
3	功耗	**未测量**	无功率计、板载无 PMBus。不能用资源反推
4	同板纯 CPU 基线给出了**不利**结论	**已补上（2026-10-29）**	A9 7.470 ms vs PL 20.790 ms ⇒ **A9 比 PL 快 2.78×**，见 §5.4。这条基线否证了「大层进 PL 更快」，不是一条好消息。保留意见：标量解释器、无 NEON，故 2.78× 是 A9 优势的下限
5	200 MHz 下时序收敛	**失败（−0.736 ns）**	累加器到 DSP 输入无流水级；修复会改变拍数，使全部已验证数字失效
6	实时情绪识别	**不成立**	0.34–0.40 对 33.3 % 机会线，接近机会
7	在线两试次校准有效	**未观察到**	RF +0.77 pt [−0.91, +2.51]、R2G +0.83 pt [−1.70, +3.50]，均含 0
8	时间差注入提升精度	**无增益**	ψ 消融−0.05 pt；打乱对照甚至更高（+0.58）
9	M=5 扩展	**当前不可行**	scratch 8,192/8,192 已满，irom 492/512
10	2025.2/2026.1 工具链重建	**未测试**	全部证据由 2024.1 产生
11	干净环境重建位流	**未验证**	Tcl 仍按开发者树布局，提交树里只能报错退出

7.2 根因分析

（1）最重要的一条：精度天花板不是硬件造成的。把引擎做得更快、更准、更多模态，都不会让 0.34 变成 0.7。AFFEC 上的三分类在被试不交叉协议下，本设计范围内的模型都落在 0.33–0.40。真正的瓶颈在特征与协议，不在实现。**把工程做得更好不等于把问题解决。**

（2）参考基线本身很高。刺激不交叉协议下，仅凭「视频身份」多数类就能拿到 0.79（`p_emotion_v`）。这说明数据里存在强烈的刺激-标签关联。但`f_emotion_a` 反过来（0.399 vs 0.480），所以不能简单说「性能全是视频泄漏」——两个协议的差值受混杂影响，不是配对因果。**这条限制没法在现有数据上消除，只能如实写。**

（3）在线校准的零结果是可信的，不是设计不力。两套独立实现（RF 与 R2G）、两套独立 bootstrap，都含 0，方向还不一致。在这种情况下继续调校准算法，是在拟合噪声。

（4）200 MHz 失败是取舍，不是疏忽。修流水级需要：改 RTL →重新导出金标准向量→重跑三档仿真→重建三份位流→重跑板上验收→更新全部 md5。在距提交 6 天时，这是「用可用换理论」的错误交易。

（5）可复现性曾是最大风险，已从「悬空」变成「可执行」。提交前，盘符在 F/G/H 之间漂移，126 处必需脚本硬编码`F:/FPGA`，直接跑会失败。现在：必需文件的 P0 硬编码为 **0**，三档仿真在新布局里全部 ALL PASS。剩下的风险是`build/*.tcl` 的树布局依赖——它已被显式 guard 拦住并写进文档。

7.3 如果继续做

优先级	动作	收益
~~ 高 ~~ 已完成	在 A9 上写 INT8 C 参考实现，同一输入逐位对齐并计时	分母补上了，但**结论是 A9 快 2.78×**，不是 PL（§5.4）。它把下一行从「可选优化」变成了「先决条件」
高	接一路真实 DRDY（EEG 或 GSR），把 `SRC=0xF` 改成真实采样沿	从「回放」变成「采集」，应用价值实质提升
中	借功率计补四态功耗（S0/S1/S2/S3）	硬缺口，有仪表即可补
~~ 中 ~~ 已完成	把`build/*.tcl` 转成扁平布局，在干净目录重建位流并比对 md5	已补上：2026-10-29 干净重建 M=4，位流逐字节相同（仅头部 5 字节时间戳），利用率/时序报告一致，见 §5.8 与 `BITSTREAM_MANIFEST.md`
高	累加器加一级流水收住 200 MHz，再提高每拍 MAC 数（多路 DSP 并行、压低 requant 开销）	§5.4 之后这条从小优化变成先决条件：只收 200 MHz 得到 10.40 ms，仍落后 A9 1.39×；要真正反超得改吞吐。代价是全部已验证拍数与向量作废，需整轮重测
低	加深`mem`/`irom` 以支持 M=5	只有在前述精度问题有解之后才有意义

7.4 复现说明与提交物对照

赛制要求	本仓库对应	状态
完整工程源码	`src/rtl`（17 个.v）、`src/model`（10 个.py）	完整
仿真/验证结果	`sim/`（8 个测试台、三档金标准向量、一条命令跑通）	可复现
综合实现报告（资源/时钟/关键性能）	`build/reports/*.rpt`、`metrics/resource.csv`、`metrics/timing.csv`	完整
上板脚本 + 实测 + 参考比对	`board/`、`metrics/latency.csv`、`metrics/cpu_baseline.csv`、`review` 结果 JSON	完整；2026-10-29 在板上重跑复验（3/3 轮 ALL PASS，中位 21.644 ms）并新增同板 CPU 基线
可复现构建脚本	`build/make_tree.ps1` + `build/*.tcl` + `build/README.md`	**已验证（2026-10-29，三档）**：仅用仓库文件重建，M=4 与 M=2 的位流与已发货版逐字节相同（只有`.bit` 头部日期不同，抹掉后 md5 相等），M=3 复现交付设计的 7,135 LUT / −0.656 ns / 197 端点（位流有 57 字节布线差异，已如实记录）
`skill/` 技能包	`skill/`（离线 50 项测试，`sh skill/tests/run_tests.sh`）	离线完整；板上模块未在硬件验证，已声明

续下页

赛制要求	本仓库对应	状态
设计报告（含大模型协作与失败分析）	本文 PDF 17 页 + `report/ai_collab.pdf` 5 页；`report/*.md` 为源，`python report/make_pdf.py` 重建	完整，可重建
开源仓库、MIT	`LICENSE`、`README.md`	完整
英文文件名/目录名	全部	符合
功耗	—	**未测量，已声明**

附：一句话总结

本作品把「多模态时间对齐 + 定点融合推理」在 PYNQ-Z2 上做成了**确定性、可逐位验证、延迟可算到单拍**的一条通路：28 个数组 5,504 个字逐位一致，一窗 21.650 ms 无宿主完成，10 ns 时间戳量化，资源 6,995 LUT / 44 BRAM / 20 DSP。它**没有**解决情绪识别精度问题，**没有**接入真实传感器，**没有**测功耗——这三条都写在第 7 节，而不是藏起来。